

2. Exam Task

The following task is part of the final project which has to be submitted until August 20, 2025. Details on how to submit will be provided later.

(1) Customizable CH (CCH)

Implement CCH with nested dissection based contraction order. This requires the following steps:

- Utilize a library that is designed to find small balanced node separators in a given graph. It is recommended to use KaHIP for this purpose¹. But other libraries or software packages can also be used (or an own implementation is fine, too, but might be a lot of additional work).
- Compute a separator decomposition by extracting a separator in the road network and recursing on the resulting subgraphs. Proceed until the final subgraphs consist of single nodes. Then derive a contraction order from the nested dissection as described in the lecture. Within a separator, an arbitrary node ordering can be chosen.
- Contract the nodes in the resulting order to build a CCH data structure. Remember to store lower triangles with each edge/shortcut.
- Implement the customization algorithm.

Conduct the following experiments and evaluations:

- Run the CCH preprocessing algorithm on all road networks in the provided data set. Measure for each graph the running time for obtaining the separator decomposition and for the actual contraction phase. Further, measure the number of shortcuts that have been inserted and the average and maximum number of lower triangles per edge. Create a plot or table with the respective results and compare the preprocessing time and shortcut count to the results for the classical CH.
- Run the customization algorithm once with the original edge weights provided in the files and 5 times with random edge weights that should be chosen uniformly at random from the interval $(0, 1)$. Measure the customization time for each run and report the time for the original average as well as the average over the 5 runs with random edge weights.
- After customization with the original edge costs, select at least 100 random source-target pairs in each graph instance, and run Dijkstra in the original graph as well as the CH-Dijkstra in the customized CCH graph. Ensure that the resulting path costs are the same. Measure the average query time and compare it to that of the CH-Dijkstra on the CH-graph.

¹<https://github.com/KaHIP/KaHIP>